

EXPERIMENT 10

Explore and implement Autoencoders for image denoising

Autoencoders

Autoencoders are neural networks for unsupervised learning that compress input data into a low-dimensional space (using an encoder) and then reconstruct it (using a decoder), training the network to minimize the reconstruction error between the original input and its reconstructed output. If the hidden layer is too large, autoencoders may simply learn to replicate the input perfectly, functioning as an identity mapping and failing to extract meaningful features.

- Denoising autoencoders address this by providing a deliberately noisy or corrupted version of the input to the encoder, but still using the original, clean input for calculating loss.
- This trains the model to learn useful, robust features and reduces the chance of simply replicating the input.

Architecture of DAE

The denoising autoencoder (DAE) architecture resembles a standard autoencoder and consists of two main components:

Encoder

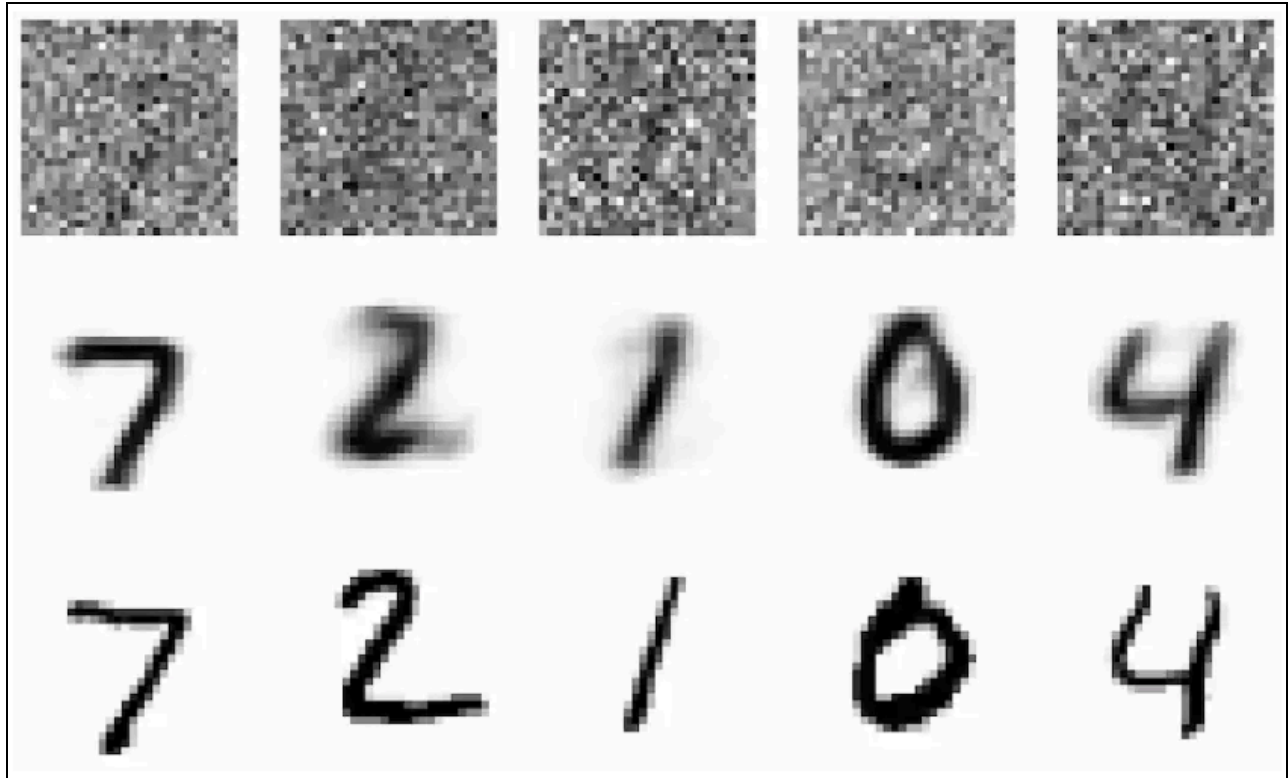
- A neural network (one or more layers) that transforms noisy input data into a lower-dimensional encoding.
- Noise can be introduced by adding Gaussian noise or randomly masking/missing some inputs.

Decoder

- A neural network (one or more layers) that reconstructs the original data from the encoding.
- The loss is calculated between the decoder's output and the original clean input, not the noisy one.

Applications of DAE

- Image Denoising: Removing noise from images to restore clear, high-quality visuals.
- Data Imputation: Filling in missing values or reconstructing incomplete data entries.
- Feature Extraction: Learning robust features that improve performance for tasks like classification and clustering.
- Anomaly Detection: Identifying outliers by measuring reconstruction errors on new data.
- Signal and Audio Denoising: Cleaning noisy sensor or audio signals, such as in speech or biomedical recordings



1.Dataset Source

Dataset: MNIST Handwritten Digits Dataset

Source: Built-in dataset available in TensorFlow (Keras)

Link (reference):

https://www.tensorflow.org/api_docs/python/tf/keras/datasets/mnist

The dataset is directly loaded using Keras, so no manual download is required.

2.Dataset Description

The MNIST dataset is a widely used benchmark dataset in image processing and deep learning.

- **Type of Data:** Image (Grayscale)
- **Total Samples:** 70,000 images
 - Training: 60,000
 - Testing: 10,000
- **Image Size:** 28×28 pixels
- **Pixel Range:** 0 to 255

- **Classes:** Digits (0–9)

Input and Output in this Experiment

- **Input:** Noisy images
- **Output:** Clean (denoised) images

Unlike classification tasks, autoencoders do not use class labels. Instead, they learn to reconstruct the original input.

Key Characteristics

- Simple and clean dataset
- Suitable for image reconstruction tasks
- Ideal for demonstrating denoising techniques

3. Mathematical Formulation of Autoencoder

An Autoencoder consists of two main functions:

Encoder Function

$$h = f(x) = \sigma(Wx + b)$$

Where:

- (x) = input image
- (h) = latent representation
- (W) = weights
- (b) = bias
- (\sigma) = activation function

The encoder compresses the input into a lower-dimensional representation.

Decoder Function

$$x' = g(h) = \sigma(W'h + b')$$

Where:

- (x') = reconstructed image

The decoder reconstructs the original image from compressed data.

Loss Function (Mean Squared Error)

$$Loss = \frac{1}{n} \sum (x - x')^2$$

Objective:

Minimize the difference between original image (x) and reconstructed image (x')

4. Algorithm Limitations

Autoencoders have several limitations:

1. Data Dependency

Requires a large dataset to learn meaningful features.

2. Overfitting Risk

Models may memorize images instead of learning patterns.

3. Loss of Details

Reconstructed images may appear blurry due to compression.

4. Computational Cost

Training deep autoencoders requires high computational resources.

5. Not Suitable for All Tasks

Basic autoencoders are not ideal for:

- Classification tasks
- Complex feature extraction without modification

6. Sensitive to Noise Level

If noise is too high, reconstruction quality decreases.

5. Methodology / Workflow

The implementation of the autoencoder follows these steps:

Step 1: Dataset Loading

Load MNIST dataset containing handwritten digit images.

Step 2: Data Preprocessing

- Normalize pixel values (0–1)
- Prepare data for model input

Step 3: Adding Noise

- Add random noise to images
- Create noisy input and clean output pairs

Step 4: Model Building

Construct Autoencoder:

- Encoder → compresses image
- Decoder → reconstructs image

Step 5: Model Compilation

- Optimizer: Adam
- Loss Function: MSE

Step 6: Model Training

Train model using:

- Noisy images (input)

- Clean images (target)

Step 7: Image Reconstruction

Use trained models to generate denoised images.

Step 8: Visualization

Compare:

- Noisy images
- Original images
- Reconstructed images

Step 9: Evaluation

Evaluate using MSE and visual inspection.

Workflow Summary

Noisy Images → Encoder → Latent Space → Decoder → Clean Images

6. Performance Analysis

The model performance is evaluated using:

1. Mean Squared Error (MSE)

Measures difference between original and reconstructed images.

- Low MSE → Good performance
- High MSE → Poor reconstruction

2. Visual Analysis

Images are compared visually:

- Noisy vs Original vs Denoised

Good model:

- Removes noise
- Preserves digit structure

Interpretation

- Loss decreases during training
- Reconstructed images are clearer
- Model successfully learns denoising

7. Hyperparameter Tuning

Hyperparameters control model performance.

Parameters Tuned

- Number of neurons
- Latent space size
- Epochs
- Batch size
- Learning rate

Tuning Process

Different combinations are tested:

Example:

- Epochs: 5, 10, 20
- Neurons: 32, 64, 128

Best combination is selected based on:

- Lowest loss
- Best image reconstruction

Impact of Tuning

- Improves image clarity
- Reduces reconstruction error
- Prevents overfitting
- Enhances model stability

Conclusion

The Autoencoder model was successfully implemented to remove noise from images and reconstruct clean outputs. The model effectively learned compressed representations and demonstrated strong performance in denoising tasks. Hyperparameter tuning further improved the quality of reconstruction, making autoencoders a powerful tool for image processing applications.